מעבדה 3: חוטים (Threads) ב־Java

הנדסת תוכנה, חורף תשפ״ז

הזדהות מאובטחת עם חוטים (נוסח לסטודנטים)

הנדסת תוכנה, חורף תשפ״ז, תרגיל מעבדה מספר 3, הגשה בזוגות

מטרות

הכרות עם חוטים ב־Java ושילובם בתוכנית ה־Login ממעבדה 2 ליצירת תהליך הזדהות מלא. בסיום התרגיל תדעו:

- ליצור חוטים בשתי הדרכים (ירושה מ־`Thread` ומימוש `Runnable`) ולהסביר מתי בוחרים בכל אחת.
- להפעיל חוט, להמתין לו (`join`), להרדים אותו (`sleep`) ולהפסיק אותו (`interrupt`).
- לזהות מצב משותף, להגן עליו בעזרת `synchronized`, ולהוכיח שההגנה עובדת.
- להעביר עבודה בין חוטים בעזרת `BlockingQueue`, בלי busy wait.
- לבצע עבודה ארוכה בלי להקפיא את ממשק JavaFX, ולעדכן את המסך רק מחוט הממשק.

רקע

תוכנית ה־Login ממעבדה 2 מאפשרת ניחוש סיסמאות ללא הגבלה. נוסיף מנגנון: אחרי n הזדהויות שגויות של אותה משתמשת היא נחסמת למשך t שניות. הלוגיקה תרוץ בחוטים נפרדים, כדי שהמסך ימשיך להגיב וכדי שכמה הזדהויות במקביל לא יפגעו בנכונות הספירה.

כללי המדיניות (חובה לכבד אותם)

1. **פרמטרים:** n (מספר הזדהויות שגויות מרבי) ו־t (משך החסימה בשניות) הם מספרים שלמים גדולים או שווים ל־1. הם מועברים בשורת ההרצה, ולא נכתבים קבועים בקוד.
2. **מי נספרת:** מעקב אחר ניסיונות מתבצע רק עבור אימייל שקיים בקובץ המשתמשות. אימייל שאינו קיים מקבל אותה הודעת שגיאה כמו סיסמה שגויה, בלי מעקב.
3. **מה נחשב כשלון:** אימייל קיים, סיסמה שגויה, והמשתמשת אינה חסומה. ניסיון בזמן חסימה אינו נספר ואינו נבדק: הוא נדחה מיד, גם אם הסיסמה נכונה.
4. **חסימה:** כשלון מספר n מתחיל חסימה של t שניות. ההודעה מציינת כמה שניות נותרו.
5. **סיום חסימה:** כשהזמן עובר המונה מתאפס ומותרים n ניסיונות חדשים, וחוזר חלילה. הזדהות מוצלחת מאפסת את המונה.
6. **זיכרון:** המצב נשמר בזיכרון בלבד ונעלם עם סגירת התוכנית.
7. **לפי אימייל:** החסימה חלה על האימייל ולא על החלון. שני חלונות, או שתי הזדהויות במקביל, אינם עוקפים את המגבלה.

דרישות

א. הכנה וארגומנטים

1. פתחו branch חדש בשם `lab3` מתוך הענף של מעבדה 2. בצעו commit בכל שלב (לפחות ארבעה commits עם הודעות משמעותיות).
   חפשו בגוגל כיצד לראות commit. צרפו תמונות מסך.
2. התוכנית מקבלת כדי ההרצה שני ארגומנטים: n ואחריו t. לדוגמה: `java -jar app.jar 3 30`. ב־IDE מזינים אותם בשדה Program arguments. בדקו אותם לפני פתיחת המסך הראשון. אם הם חסרים, אינם מספרים שלמים או קטנים מ־1, בקשו מהמשתמשת להזין אותם שוב ידנית.

ב. החוטים

חוט	סוג	תפקיד
חוט הממשק של JavaFX	קיים מראש	מציג ומעדכן רכיבי מסך בלבד. אסור לבצע בו קריאת קובץ, `sleep` או המתנה.
`LoginTask`	`Runnable`	נוצר בכל לחיצה על Login ומנהל את תהליך ההזדהות. מחזיר תוצאה למסך דרך `Platform.runLater`. (למה כדאי?)
`BlockCheckThread`	ירושה מ־`Thread`	בודק אם האימייל חסום. `LoginTask` מפעיל אותו וממתין לו ב־`join`.
`FailureWorker` (שניים)	ירושה מ־`Thread`, daemon	צורכים אירועי הזדהות שגויה מ־`BlockingQueue`, מעדכנים את המונה וחוסמים כשמגיעים ל־n .
`CountdownTask`	`Runnable`	ספירה לאחור חיה במסך בזמן חסימה.

ג. מצב משותף וסנכרון

- מחלקה `UserState` לכל אימייל, עם מונה כשלונות ושעת סיום החסימה. כל המתודות שלה `synchronized`.
- מחלקה `AttemptTracker` שמחזיקה `ConcurrentHashMap` מאימייל ל־`UserState`, ומקבלת n ו־t בבנאי.
- בדיקה בעזרת מחלקה נוספת שתפועל בעזרת לאחר לחיצה על כפתור נוסף על מסך הכניסה. שמריצה 20 חוטים במקביל נגד אותו אימייל ומוכיחה שלא אבדו עדכונים. עם n=10,000: כל חוט מבצע 499 כשלונות (9,980 בסך הכול). אחר כך 19 כשלונות נוספים אינם חוסמים, והכשלון ה־20 כן חוסם.
- ניסוי: הריצו את הבדיקה פעם עם `synchronized` ופעם בלעדיו (לצורך הניסוי בלבד, ואז החזירו). צרפו את שתי התוצאות לקובץ ההסבר.

ד. ממשק המשתמש

- בזמן שהתהליך רץ, שדות הקלט וכפתור Login מושבתים כדי למנוע לחיצות כפולות, והחלון נשאר רספונסיבי (אפשר להזיז אותו).
- הודעות: הזדהות שגויה עם אימייל קיים מציגה כמה ניסיונות נותרו. אימייל לא קיים מציג "פרטים שגויים" בלבד. חסימה מציגה "החשבון חסום, נותרו X שניות" שמתעדכן כל שנייה. בסיום החסימה ההודעה נעלמת והכפתור נפתח.
- הצלחה פותחת את מסך ה־Welcome של מעבדה 2.
- סגירת החלון באמצע ספירה מסיימת את כל החוטים והתוכנית נסגרת. ודאו ב־IDE שהתהליך אכן נעצר.

זרימת ההזדהות (חובה)

1. לחיצה על Login: ה־controller משבית את הכפתור ומפעיל חוט חדש עם `LoginTask`.
2. `LoginTask` מפעיל `BlockCheckThread` לאימייל וממתין לו ב־`join`. אם האימייל חסום, התוצאה היא "חסום" והתהליך מסתיים, בלי לבדוק סיסמה.
3. אחרת `LoginTask` מאמת את הפרטים בעזרת הקוד של מעבדה 1. הצלחה: איפוס המונה והתוצאה "הצלחה".
4. כשלון עם אימייל שאינו קיים: התוצאה "שגיאה" בלי מעקב.
5. כשלון עם אימייל קיים: `LoginTask` מניח אירוע `FailedAttempt` בתור (`put`) וממתין (`take`) לתשובה מ־`FailureWorker`, שמעדכן את המונה וקובע אם נותרו ניסיונות או שהמשתמשת נחסמה.
6. `LoginTask` שולח את התוצאה ל־controller בעזרת `Platform.runLater`. במקרה של חסימה ה־controller מפעיל `CountdownTask`.

> שימו לב: `FailedAttempt` מכיל גם תור קטן (`ArrayBlockingQueue` בגודל 1) שדרכו `FailureWorker` מחזיר תשובה. כך `LoginTask` יודע מיד אם הכשלון הזה גרם לחסימה.

שאלות:

1. למה אסור להמתין t שניות על חוט ה־JavaFX? מה רואה המשתמשת, ואיך הפתרון שלכם נמנע מכך?
2. איזו בעיה עלולה לקרות ב־`recordFailure` בלי `synchronized`? הדגימו עם תוצאות הניסוי שלכם.
3. למה להשתמש ב־`BlockingQueue` ולא ב־`ArrayList` עם לולאת busy wait?
4. עבור כל חוט בטבלה בסעיף 3.4 ב׳, הסבירו למה נבחר דווקא `Runnable` או ירושה מ־`Thread`.

הוראות הגשה

1. קובץ PDF שכולל: קישור לתיקיית ה־repo ב־GitHub (ענף `lab3`); הסבר על כל מחלקה וחוט; תשובות לשאלות בסעיף 3.6, צילומי מסך מהרצה תקנית במחשבכם: (א) הרצה עם ארגומנטים שגויים, (ב) הזדהות מוצלחת, (ג) שגיאה עם מספר ניסיונות שנותרו, (ד) חסימה עם ספירה לאחור, (ה) שחרור אחרי t שניות.
2. כל קבצי המקור של התוכנית כפרויקט שמתקמפל (בGitHub).
3. ההגשה כרגיל בזוגות, בקובץ PDF בלבד, ושם הקובץ הוא תעודות הזהות מופרדות ב־"_".

הזדמנות לאינטגרציה עם מעבדות 1–2:

- אימות האימייל והסיסמה נעשה במחלקת השירות של מעבדה 1. היא זורקת חריגה בקלט לא תקין, ואתם תופסים אותה ב־`LoginTask`.
- הגדירו מחלקה `LoginResult` (הצלחה, שגיאה עם מספר הניסיונות שנותרו, או חסום עם הזמן שנותר) שדרכה `LoginTask` מדווח ל־controller.
- ה־controller של מעבדה 2 נשאר דק: קורא שדות, מפעיל `LoginTask` ומציג את התוצאה. אסור שתהיה בו לוגיקה של ספירה או של זמן.

חומרי עזר:

- יצירה והפעלה של חוטים:
  https://jenkov.com/tutorials/java-concurrency/creating-and-starting-threads.html
- התיעוד הרשמי של Oracle למחלקה Thread:
  https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html
- מנגנון נעילה מתקדם:
  https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/Lock.html